Práctica: Especificación Formal e Implementación de un Servicio Cliente-Servidor sobre Red IP mediante Programación con Sockets

Protocolo de streaming continuo con control de flujo y gestión de clientes lentos

Carmen Segura Ruiz, Ismael Ropero Ramírez

carmenruizz@correo.ugr.es, ismaelropero@correo.ugr.es

ÍNDICE

ÍNDICE 3
Especificación formal del protocolo 4
1.1 Mensajes de Datos (Tipo Msg) 4
1.2 Mensajes de Control (Tipo CTRL) 4
1.3 Definición ABNF 5
Diagrama de estados del servidor y del cliente 7
Diagrama de secuencia 8
1.1. Inicio de sesión 8
Alta (autenticación) 8
1.2. Envío de datos 8
Envío de datos y ACK 8
1.3. Pérdida mensaje (Cliente lento) 9
Modo lento — activado por timeout de ACK 9
Modo lento — toggle manual (tecla 's' en el cliente) 9
1.4. Desconexión 10
1.4.1. Exitosa 10
1.4.2. No exitosa 10
1.5. Pausa 11
Pausa y reanudación 11
Justificación técnica 11
Organización del código 12

Especificación formal del protocolo

El protocolo diseñado en esta práctica es un protocolo de streaming continuo con control de flujo y gestión de clientes lentos.

El protocolo de aplicación diseñado funciona sobre TCP y se encarga de la transmisión fiable y controlada de un flujo de texto entre un servidor (emisor) y uno o varios clientes (receptor). La comunicación multiplexa datos y control en el mismo canal utilizando cadenas de texto (Strings) codificadas con primitivas writeUTF y readUTF de Java.

El protocolo define dos tipos de mensajes.

1.1 Mensajes de Datos (Tipo Msg)

El servidor envía el flujo de texto palabra a palabra. Cada mensaje de datos tiene el formato:

```
Msg <número-secuencia> <palabra>
```

- número-secuencia: Entero incremental desde 0. El cliente lo usa para detectar pérdidas o mensajes fuera de orden.
- palabra: Token de texto sin espacios. El texto fuente se parte por espacios antes de enviarse.

Para la demostración, las palabras han sido sacadas de un texto del Quijote, que son las que se envían de forma continua desde el servidor a los clientes.

1.2 Mensajes de Control (Tipo CTRL)

Por otro lado tenemos los mensajes de control, con sintaxis

```
"CTRL <código> [parámetros]".
```

A continuación especificamos los distintos mensajes de control:

- CTRL 0 : Petición de autenticación enviada por el cliente al iniciar la conexión
- CTRL 1: Aceptación o confirmación del modo lento.
- CTRL 2 : Confirmación de autenticación exitosa (servidor al cliente)
- CTRL 3: Confirmación de autenticación fallida (servidor al cliente)
- CTRL 4: Mensaje ACK enviado por el cliente cada 20 palabras recibidas.
- CTRL 5: Aviso de modo lento. Se puede originar por el servidor (si detecta un timeout de ACK) o solicitado de forma manual por el cliente.
- CTRL 6: Petición de cierre que origina el cliente.
- CTRL 7: Señal de fin de la transmisión y cierre originada por el servidor (tanto por desconexión como por haber enviado todo el texto).
- CTRL 8: Petición de parar el flujo de datos (del Cliente al Servidor)
- CTRL 9: Petición de reanudar el flujo de datos (Cliente al Servidor)

1.3 Definición ABNF

A continuación se muestra la definición formal de los mensajes y sus campos utilizados en este protocolo mediante el uso de ABNF:

```
;---- Estructura general

mensaje = control / datos / error


;---- Control

control = "CTRL" SP codigo_control SP [user]

codigo_control = %x30-x39 ; Los codigos de control van del 0 al 9

user = <1> ALPHA <6>; usuario con el que se autentica en el mensaje de ID (CTRL 0)


;---- Datos

datos = "Msg" SP numero_secuencia SP [signo_puntuacion] palabra 1*3[signo_puntuacion]; los datos pueden contener signos de puntuacion delante o detras de la palabra. El maximo por delante es 1 y el maximo por detras es 3 debido a los puntos suspensivos.

numero_secuencia = 0" / (POS-DIGIT *9DIGIT); el numero de secuencia no puede ser negativo. El valor maximo es el de un int, 2147483647 = 2^31-1 al tomar solo positivos.

POS-DIGIT = %x31-39


palabra = <1>ALPHA / enie / vocal_tilde / vocal_dieresis<255>; Usamos un numero tan alto con el objetivo de recopilar incluso términos cientificos en ingles y español

vocal_tilde = %xC1 / %xE1 / %xC9 / %xE9 / %xCD / %xED / %xD3 / %xF3 / %xDA / %xFA ; "ó" / "Ó" / "á" / "Á" / "é" / "É" / "í" / "Í" / "ú" / "Ú"

vocal_dieresis = %xE4 / %xEB / %xEF / %xF6 / %xFC / %xC4 / %xCB / %xCF / %xD3 / %xDC; "ä" / "ë" / "ï" / "ö" / "ü" / "Ä" / "Ë" / "Ï" / "Ó" / "Ü"

enie = %xD1 / %xF1; "ñ" y "Ñ"

signo_puntuacion = punto / coma / punto-y-coma / dos-puntos / guion / comilla-simple / comilla-doble / exclama-cierra / interroga-cierra / exclama-abre / interroga-abre / pare ntesis-abre / parentesis-cierra / corchete-abre / corchete-cierra / raya-guion-bajo
```

```
;---- Signos Básicos ---
punto           = %x2E    ; .
coma            = %x2C    ; ,
punto-y-coma    = %x3B    ; ;
dos-puntos      = %x3A    ; :
guion           = %x2D    ; -
comilla-simple  = %x27    ; '
comilla-doble   = %x22    ; "
---- Signos de Exclamación e Interrogación ---
exclama-cierra  = %x21    ; !
interroga-cierra = %x3F    ; ?
exclama-abre    = %xA1    ; ¡
interroga-abre  = %xBF    ; ¿
---- Paréntesis y Corchetes ---
parentesis-abre  = %x28    ; (
parentesis-cierra = %x29    ; )
corchete-abre    = %x5B    ; [
corchete-cierra   = %x5D    ; ]
---- Otros Signos de Escritura ---
puntos-suspensivos = punto punto punto ; ...
raya-guion-bajo    = %x5F           ; _
```

1.4 Errores

Internamente ambos nodos muestran distintos mensajes de error cuando les ocurre algo inesperado. Sin embargo, no comunican dichos errores al otro nodo, si no que cierran la comunicación con los mensajes de control antes expuestos.

Estos son:

Código	Mensaje	Descripción	Reacción del receptor
ERR 0	Err 0	Formato de mensaje incorrecto (no cumple la gramática ABNF del protocolo).	El receptor cierra la conexión (envía CTRL 7 si es el servidor, o CTRL 6 si es el cliente).
ERR 1	Err 1	Usuario o contraseña incorrectos durante el alta.	El servidor responde con CTRL 3 y cierra la conexión.
ERR 2	Err 2	Opción o comando inexistente.	El receptor responde con CTRL 7 (cierre) para indicar que no puede continuar.
ERR 3	Err 3	Cliente demasiado lento (el servidor detecta timeout de ACK).	El servidor envía CTRL 5 para activar modo lento y espera CTRL 1 del cliente.
ERR 4	Err 4	Comando no permitido en el contexto actual (p. ej. el cliente intenta enviar datos).	Se ignora. El receptor no toma ninguna acción.
ERR 5	Err 5	El cliente intenta autenticarse de nuevo estando ya en sesión.	Se ignora. El receptor no toma ninguna acción.

Diagrama de estados del servidor y del cliente

Diagrama de estados del servidor:

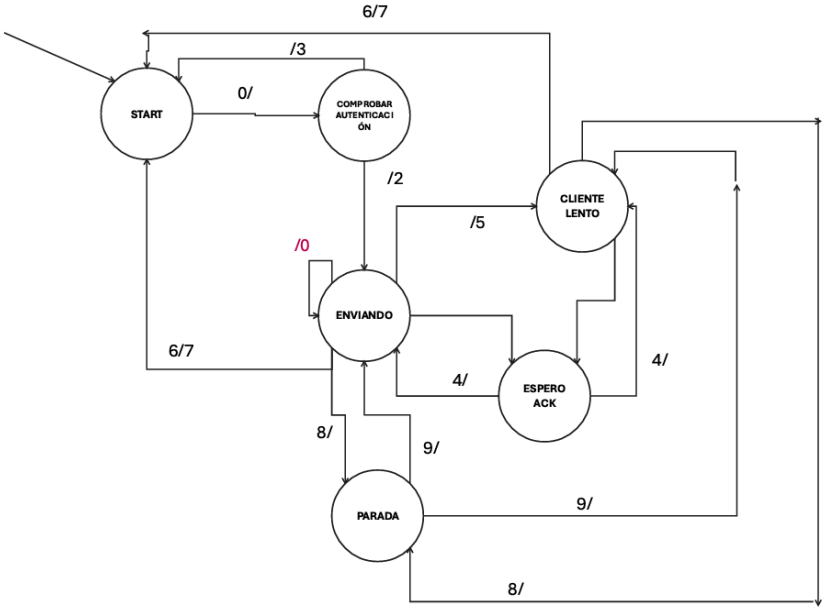


Diagrama de estados del cliente:

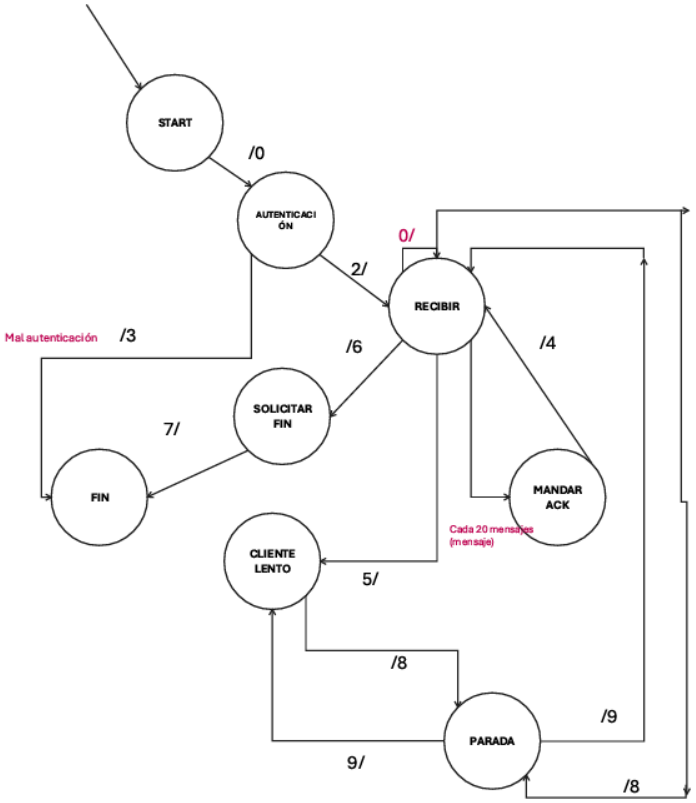
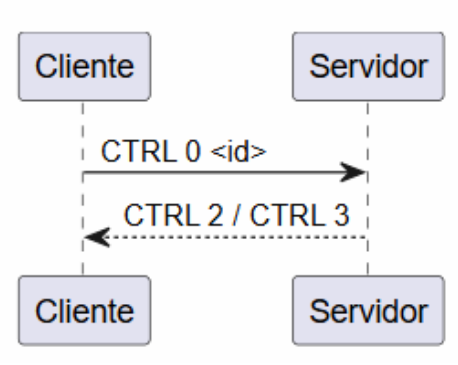


Diagrama de secuencia

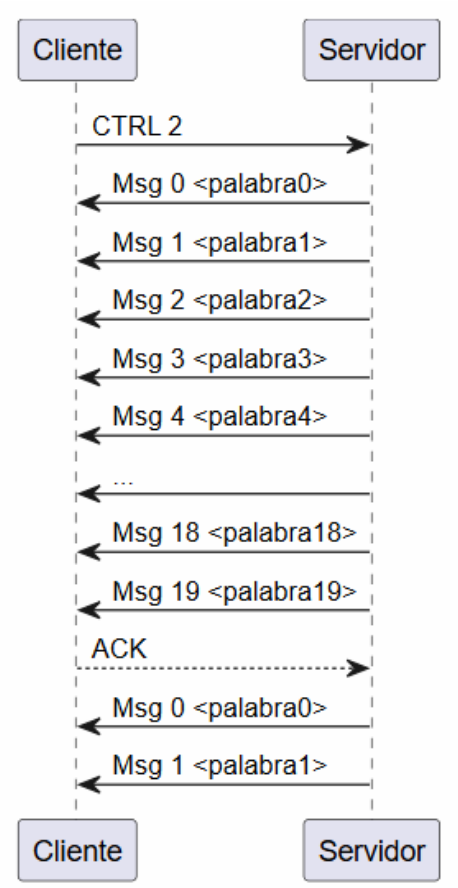
1.1. Inicio de sesión



Alta (autenticación)

- **Cliente** → **Servidor:** CTRL 0 <id>

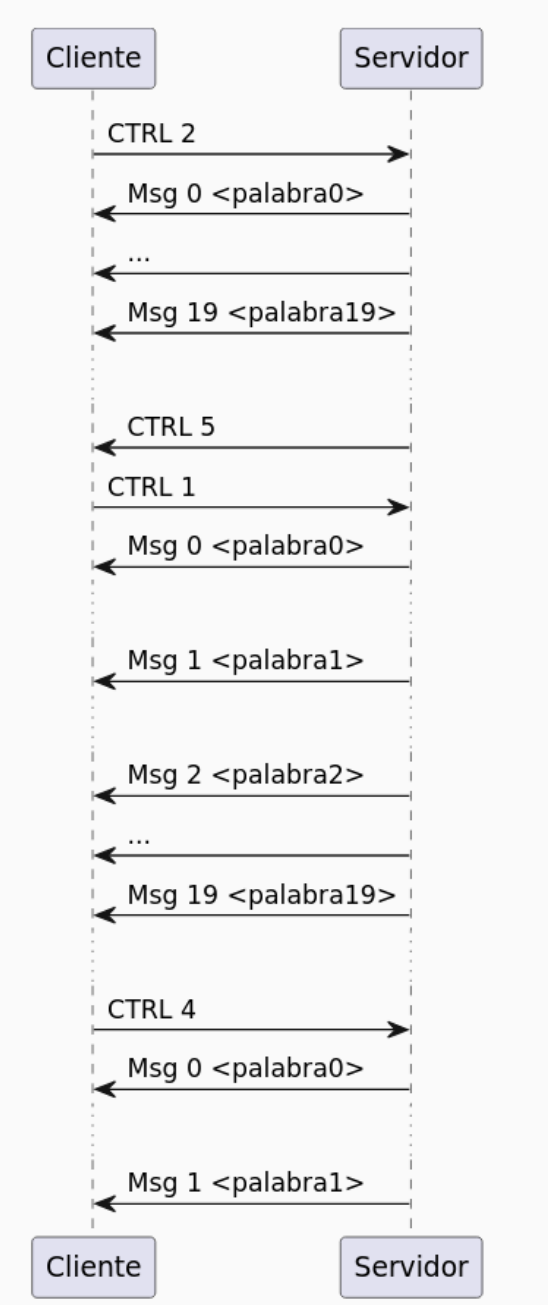
1.2. Envío de datos



Envío de datos y ACK

- **Servidor** → **Cliente:** Msg <seq> <palabra> (bloque de 20 mensajes)
- **Cliente** → **Servidor:** CTRL 4 (ACK al completar el bloque)

1.3. Pérdida mensaje (Cliente lento)



Modo lento — activado por timeout de ACK

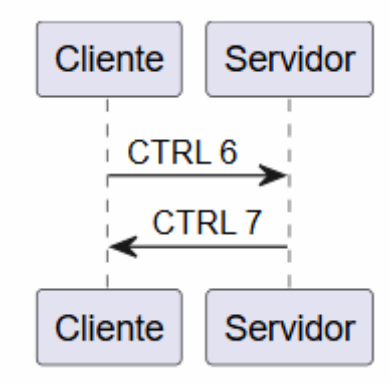
- **Servidor → Cliente:** CTRL 5 (aviso de cliente lento)
- **Cliente → Servidor:** CTRL 1 (aceptación)
- **Servidor → Cliente:** CTRL 1 (confirmación; el servidor aplica retardo entre palabras)

Modo lento — toggle manual (tecla 's' en el cliente)

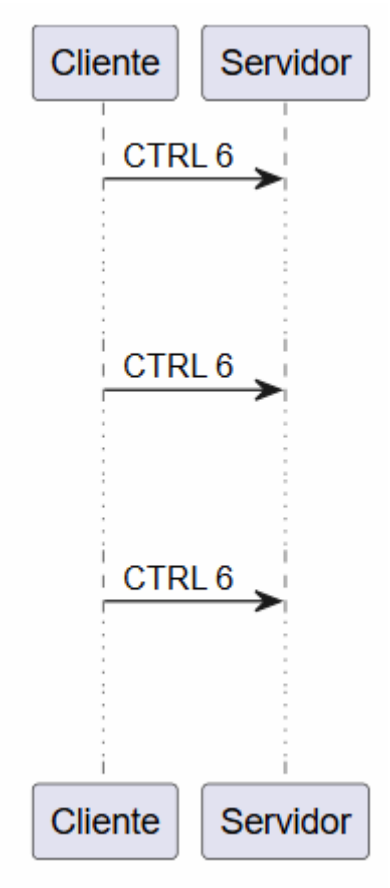
- **Cliente → Servidor:** CTRL 5 (solicitud de toggle)
- **Servidor → Cliente:** CTRL 1 (confirmación; activa o desactiva el retardo)

1.4. Desconexión

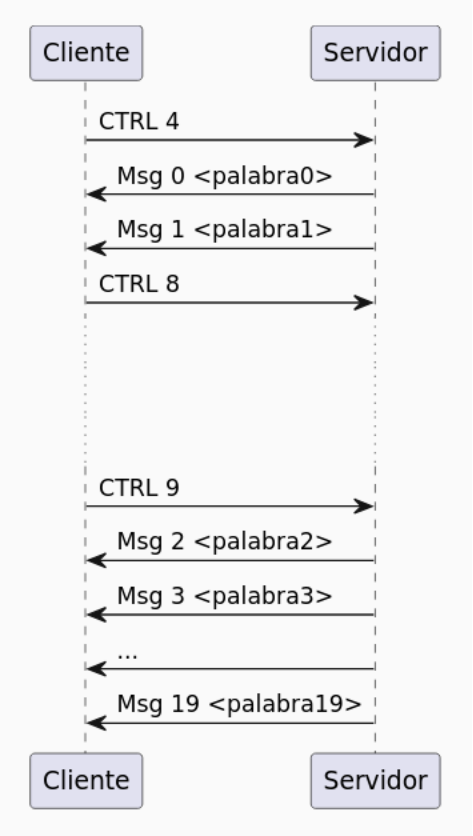
1.4.1. Exitosa



1.4.2. No exitosa



1.5. Pausa



Pausa y reanudación

- **Cliente → Servidor:** CTRL 8 (pausa)
- **Cliente → Servidor:** CTRL 9 (reanudación)

El servidor deja de enviar mensajes mientras está en pausa. El timeout de ACK se reinicia al reanudar para no penalizar al cliente.

Justificación técnica

El protocolo se ha diseñado priorizando la fiabilidad y el control de la congestión a nivel de aplicación, características que justifican el uso de TCP como protocolo de transporte subyacente. Sin embargo TCP no permite saber (por sí solo) al servidor si la aplicación de un cliente está procesando la información a la misma velocidad que llega ; es por eso que se ha implementado un mecanismo de ventana deslizante mediante ACKs (mensaje CTRL 4).

En lugar de abrir un puerto para datos y otro para el control se utiliza un único socket multiplexando los comandos con prefijos (CTRL o Msg).Esto facilita el control del estado.

Por otro lado este sistema implementa un control de flujo adaptativo, un temporizador y comandos de degradación (CTRL 5 y CTRL 1). Si el cliente no responde a tiempo,el servidor no asume una caída inmediata, sino primero asume que hay congestión, por lo que pasa a un ‘modo lento’ , induciendo retardo en las palabras.

Ya que el servidor debe seguir escuchando comandos de control (como por ejemplo CTRL 8, de pausa) mientras envía flujo, ha sido necesario en el diseño desacoplar lectura y escritura.

Constantes configuradas:

Constante	Valor por defecto	Descripción
PUERTO	5555	Puerto TCP en el que escucha el servidor.
PALABRAS_POR_BLOQUE	20	Número de palabras que el servidor envía antes de esperar un ACK.
ACK_TIMEOUT_MS	4000 ms	Tiempo máximo de espera del servidor para recibir un ACK antes de activar modo lento.
RETARDO_LENTO_MS	200 ms	Retardo entre palabras que aplica el servidor en modo lento.

Organización del código

La implementación del código sigue una arquitectura cliente-servidor dividida en dos clases principales:

La clase de servidor. java, cuyas principales funciones son:

- Gestión del ciclo de vida del ServerSocket y las conexiones que entran.
- Mantiene el estado de las transmisiones (modos lentos, pausas,etc)
- Se encarga de trocear la cadena de texto (fragmento del Quijote) y controlar el bucle principal de envío.
- Implementa la lógica de gestión de los temporizadores para la transición al modo lento y procesa comandos de estado entrantes de forma concurrente.

La clase cliente.java:

- Inicia el socket contra la IP y el puerto de escucha asignados.
- Envía la cadena de autenticación inicial.
- Implementa el bucle de recepción pasivo para capturar los mensajes Msg y procesarlos.
- Lleva la cuenta de secuencias de parada de manera autónoma, tira los “CTRL 4” (ACKs) cada vez que se alcanza el tamaño de bloque.
- A través de un hilo independiente captura entradas del usuario para enviar señales de control (pausa, modo lento, cierre) al servidor sin interrumpir el flujo principal de recepción.